RELAY — the autonomous dispatch desk

Solution overview for the engineering manager · MERN + TypeScript

95 automated tests · strict TypeScript · runs with **npm install && npm run dev**

1 PROBLEM → SOLUTION

Problem. Dispatchers juggle availability, phone calls and spreadsheets. When a rider doesn't answer or nobody fits, orders sit idle — and nobody notices until a customer complains.

- **Recommend** eligible riders with an explained ranking.
- **Reserve** rider capacity the moment someone is assigned; **release** it on any failure.
- **Auto-escalate**: no confirmation in 2 h (configurable) → job lands in the **Manual Queue**, with the reason on record.
- **Surface** what needs a human on one dashboard; **audit** every action.

2 DISPATCHER'S PRIMARY WORKFLOW

Job created › Ranked riders + "why" › Assign / override (reason) › Rider confirms › Picked up → Delivered

No answer in 2 h › Manual queue › Investigate › Reassign › Done

- Dashboard: 7 live counters, pipeline, **Needs attention** feed (critical / warning).
- Job page: lifecycle stepper, countdown ring, full activity timeline, every failed attempt kept.
- Riders page: capacity ledger + "can they take another delivery?" check.

8	1	0
job states, guarded transitions	open assignment per job (DB-enforced)	ledger drift — integrity check asserted across the tests

3 RECOMMENDATIONS & AUTOMATIC QUEUE HANDLING

Hard rules (never recommended): unavailable · pickup/delivery zone not served · not enough free capacity · no free job slot · already rejected this job.

Score 0–100	Normal	High	Urgent
Travel to pickup	35	45	55
Workload (spare slots)	30	20	10
Capacity fit (right-size)	10	5	5
Deadline headroom	25	30	30

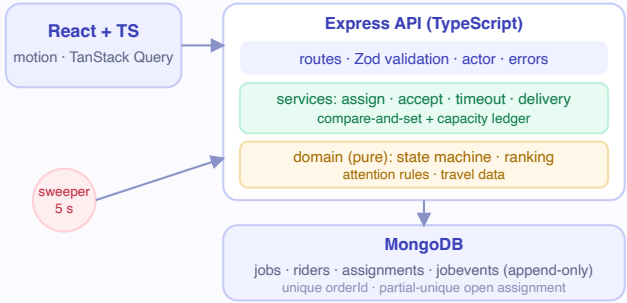
- Each candidate shows per-factor points and a sentence: *"Recommended over Rahul: 9.4 pts higher, mainly closer to pickup."*
- Picking anyone but #1 **requires a reason**; both riders & scores are logged.
- Nobody eligible → job stays **unassigned**, is flagged once, appears in "Needs attention".
- **Timeout** (sweeper + lazy check on accept + orphan repair): attempt → *EXPIRED* (kept), capacity released, job → Manual Queue, immediately re-assignable. **Reject / withdraw** take the same path.
- Reviewer can set the window to 0.5 min or **advance a simulated clock** — no 2-hour wait.

! ONE IMPORTANT TRADE-OFF

Compare-and-set instead of transactions. I made every write a single-document CAS (job status+pointer, assignment status, guarded capacity `$inc`) and order steps so a crash leaves something *detectable*, instead of requiring a MongoDB replica set.

- **Gain:** runs on any `mongod` (even in-memory), races resolved and covered by tests: accept-vs-timeout, two dispatchers, over-commit.
- **Cost:** a crash between a state change and its audit event could drop an event; mitigated by an integrity check + orphan repair, fixed properly by transactions/outbox next.

4 ARCHITECTURE & DATA MODEL



- **Job ≠ assignment:** a job accumulates attempts (timed-out, rejected, reassigned) that are never deleted.
- **Rider ledger:** reserved vs active load, changed only by whoever wins the assignment CAS → released exactly once.
- **History:** separate append-only event log (type, time, actor, details) — status never used to rebuild the past.
- Indexes: `{status, priority, createdAt}` queues · `{status, expiresAt}` sweeper · `{jobId,_id}` history.

→ IF I HAD ANOTHER WEEK

- **Transactions / outbox + lease-based sweeper** — close the last consistency gap; scale out API nodes.
- **Learned dispatch** — rider acceptance & on-time history, batch assignment across waiting jobs, offline replay to tune weights.
- **Real-time + rider channel + auth** — SSE push, reminder before timeout, JWT roles.